

Docker

Java Backend Interview — Short Notes

1. Core Concepts

Concept	Description	Analogy
Image	Read-only template with app + runtime + dependencies. Blueprint. Layered filesystem	Class (blueprint)
Container	Running instance of an image. Isolated process with own filesystem/network/PID	Object (instance)
Dockerfile	Text file with instructions to build an image. Each instruction = one layer	Build script / recipe
Registry	Store and distribute images. Docker Hub (public), ECR, GCR, ACR (private)	Maven Central for images
Layer	Each Dockerfile instruction adds a read-only layer. Shared between images — efficient	Git commit / delta
Volume	Persistent storage outside container filesystem. Survives container restart/removal	Mounted external drive
Network	Virtual network for container communication. Bridge (default), Host, None, Overlay	LAN segment
Docker Daemon	Background service managing containers, images, networks, volumes (dockerd)	JVM / runtime
Docker Client	CLI (docker) sending commands to daemon via REST API	javac / build tool
Docker Compose	Multi-container orchestration via YAML — define and run multi-service apps locally	Local kubernetes / Procfile

1.1 VM vs Container

	Virtual Machine	Container
Isolation	Full OS isolation — own kernel	Process isolation — shares host kernel
Size	GBs — full OS image	MBs — only app + libs
Startup	Minutes	Seconds or milliseconds
Overhead	High — hypervisor + guest OS	Low — namespaces + cgroups
Portability	Portable but heavy	Highly portable — runs anywhere Docker installed
Use for	Full OS isolation, different kernels, legacy apps	Microservices, CI/CD, cloud-native apps

2. Dockerfile

2.1 Dockerfile Instructions

Instruction	Description	Example
FROM	Base image — first instruction. Defines starting layer	FROM eclipse-temurin:21-jre-alpine
WORKDIR	Set working directory in container — creates if not exists	WORKDIR /app

COPY	Copy files from host to container. Preferred over ADD for local files	COPY target/app.jar app.jar
ADD	Like COPY + extracts tar, supports URLs. Use COPY for simplicity	ADD app.tar.gz /app/
RUN	Execute command during build — creates new layer	RUN apt-get install -y curl
CMD	Default command when container starts — overridable at runtime	CMD ["java", "-jar", "app.jar"]
ENTRYPOINT	Executable to run — not overridable (args appended instead)	ENTRYPOINT ["java", "-jar", "app.jar"]
EXPOSE	Document port container listens on — does NOT publish port	EXPOSE 8080
ENV	Set environment variable available in container	ENV JAVA_OPTS="-Xmx512m"
ARG	Build-time variable — not available in running container	ARG VERSION=1.0
VOLUME	Create mount point for persistent data	VOLUME ["/data"]
USER	Switch to non-root user — security best practice	USER appuser
LABEL	Add metadata to image	LABEL maintainer="team@company.com"
HEALTHCHECK	Command to check container health	HEALTHCHECK CMD curl -f http://localhost:8080/health
STOPSIGNAL	Signal to stop container (default SIGTERM)	STOPSIGNAL SIGINT

2.2 Production Dockerfile — Spring Boot

```
# Stage 1: Build
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -B # cache deps layer
COPY src ./src
RUN mvn package -DskipTests -B

# Stage 2: Runtime — slim image
FROM eclipse-temurin:21-jre-alpine
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
WORKDIR /app
COPY --from=build /app/target/app.jar app.jar
USER appuser # non-root
EXPOSE 8080
HEALTHCHECK --interval=30s --timeout=3s \
CMD wget -qO- http://localhost:8080/actuator/health || exit 1
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75.0", "-jar", "app.jar"]
```

2.3 Layer Caching Strategy

- Docker caches each layer. If layer unchanged, uses cache. Put **rarely changing** steps first
- Copy pom.xml + download deps BEFORE copying src — deps cached unless pom changes
- Spring Boot layered JAR: extract layers (dependencies, spring-boot-loader, app) for fine-grained caching

```
# Spring Boot layered JAR approach
FROM eclipse-temurin:21-jre-alpine AS extract
WORKDIR /app
```

```

COPY target/app.jar app.jar
RUN java -Djarmode=layertools -jar app.jar extract

FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=extract /app/dependencies/ ./
COPY --from=extract /app/spring-boot-loader/ ./
COPY --from=extract /app/snapshot-dependencies/ ./
COPY --from=extract /app/application/ ./ # only this layer changes on code change
ENTRYPOINT ["java", "org.springframework.boot.loader.launch.JarLauncher"]

```

3. Essential Docker Commands

Command	Description
docker build -t name:tag .	Build image from Dockerfile in current dir. -t tags it. --no-cache ignores cache
docker run -p 8080:8080 --name app img	Run container. -p host:container port. --name for easy reference. -d for detached
docker run -d -e KEY=val --env-file .env img	-d = detached (background). -e inject env var. --env-file inject from file
docker run -v /host/path:/container/path img	Bind mount — map host dir to container dir. Or -v vol-name:/path for named volume
docker ps / docker ps -a	List running containers / all containers including stopped
docker stop / docker kill container	SIGTERM graceful stop / SIGKILL force kill
docker rm container / docker rm -f	Remove stopped container / force remove running container
docker logs -f container	Stream container logs. --tail 100 for last 100 lines
docker exec -it container bash	Open interactive shell in running container. -it = interactive + TTY
docker inspect container/image	Full JSON config — networking, mounts, env vars, image layers
docker images / docker image ls	List all local images with size and tags
docker rmi image / docker image prune	Remove image / remove all dangling (untagged) images
docker pull / docker push image:tag	Pull from registry / push to registry
docker tag source target	Tag image — docker tag myapp:latest registry.io/myapp:1.0
docker stats	Live resource usage — CPU, memory, network I/O per container
docker system prune -a	Remove all stopped containers, unused images, dangling volumes. Frees disk
docker cp container:/path ./local	Copy file from container to host (or reverse)
docker network ls / inspect	List networks / inspect network config and connected containers
docker volume ls / create / rm	List/create/remove named volumes

4. Docker Compose

- Define and run multi-container applications. Single YAML file — services, networks, volumes
- Essential for local development — spin up app + DB + cache + message broker together

```

# docker-compose.yml
version: '3.9'

```

```

services:
  app:
    build: . # build from local Dockerfile
    ports: ["8080:8080"]
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/orders
      SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:9092
    depends_on:
      db: { condition: service_healthy }
      kafka: { condition: service_started }
    networks: [backend]

  db:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB: orders
      POSTGRES_USER: app
      POSTGRES_PASSWORD: secret
    volumes:
      - pgdata:/var/lib/postgresql/data # named volume – persists
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app"]
      interval: 5s; timeout: 3s; retries: 5
    networks: [backend]

  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]
    networks: [backend]
    volumes:
      pgdata: # named volume managed by Docker
    networks:
      backend:
        driver: bridge

```

Compose Command	Description
docker compose up -d	Start all services in background. --build to rebuild images
docker compose down	Stop and remove containers. -v also removes volumes
docker compose logs -f app	Follow logs of specific service
docker compose ps	Status of all services in compose file
docker compose exec app bash	Shell into running compose service
docker compose scale app=3	Scale a service to N replicas (without Swarm)
docker compose pull	Pull latest images for all services
docker compose restart app	Restart specific service

5. Docker Networking

Network Mode	Description	Use Case
bridge (default)	Private internal network. Containers communicate by name. Isolated from host network	Multi-container apps on single host. Default for docker run and compose

host	Container uses host's network directly — no network isolation. Port mapping not needed	Performance-critical apps, avoid NAT overhead. Not available on Mac/Windows Docker Desktop
none	No network — fully isolated container	Batch processing, security-sensitive jobs with no network needs
overlay	Multi-host network spanning Docker Swarm nodes. Encrypted option available	Docker Swarm multi-node deployments
macvlan	Container gets its own MAC address — appears as physical device on network	Legacy apps expecting direct LAN connection

- Containers in same network communicate by **service/container name** (DNS resolution). No need for IP addresses
- Containers in different networks cannot communicate — add to same network or use network alias

```
# Custom network – containers connect by name
docker network create mynet
docker run -d --name db --network mynet postgres:15
docker run -d --name app --network mynet -e DB_HOST=db myapp
# app container reaches db at hostname 'db'
```

6. Storage & Volumes

Storage Type	Description	Persistence	Best For
Named Volume	Docker-managed storage. <code>docker volume create</code> . Path: <code>/var/lib/docker/volumes/</code>	Survives container removal	DB data, persistent app state
Bind Mount	Map host directory into container. Full path required	Depends on host dir	Dev hot-reload, config injection
tmpfs mount	In-memory only — not written to disk	Deleted with container	Sensitive temp data, secrets in memory
Container FS	Container's writable layer — changes lost on removal	Lost on removal	Temp files during container lifetime

```
# Named volume (recommended for production data)
docker run -v pgdata:/var/lib/postgresql/data postgres:15
# Bind mount (dev – code hot reload)
docker run -v $(pwd)/src:/app/src:ro myapp # :ro = read-only
# tmpfs – sensitive data in memory only
docker run --tmpfs /app/secrets:rw,noexec,nosuid,size=65536k myapp
```

■ **Never store secrets in images or env vars in plain text. Use Docker Secrets (Swarm), Vault, or K8s Secrets**

7. Docker Security Best Practices

- **Non-root user** — add `USER` instruction. Root in container can be root on host if misconfigured
- **Minimal base image** — Alpine, distroless. Less surface area = fewer vulnerabilities
- **Multi-stage builds** — build tools not in final image. No compiler, no source code in prod image
- **Read-only filesystem** — `docker run --read-only`. Only writable where needed (`-v` for data)
- **Drop capabilities** — `--cap-drop=ALL --cap-add=NET_BIND_SERVICE`. Minimum Linux capabilities
- **Scan images** — `docker scout`, `Trivy`, `Snyk`. Detect known CVEs in base images and deps
- **No sensitive data in layers** — secrets in image layers persist even after deletion in later layer
- **Pin image versions** — FROM `postgres:15.4` not FROM `postgres:latest` — reproducible builds
- **Resource limits** — `--memory=512m --cpus=0.5`. Prevent one container starving host

```
# Secure run example
```

```
docker run \
--read-only \ # read-only root FS
--cap-drop=ALL \ # drop all Linux capabilities
--cap-add=NET_BIND_SERVICE \ # add back only what's needed
--memory=512m --memory-swap=512m \
--cpus=0.5 \
--security-opt=no-new-privileges \
--user 1001:1001 \
myapp:1.0
```

8. Registry & CI/CD

8.1 Image Tagging Strategy

- **Semantic versioning:** myapp:1.2.3 — immutable. myapp:1.2 — latest patch. myapp:latest — not recommended for prod
- **Git SHA tags:** myapp:a1b2c3d — pinpoint deployed commit. Reproducible
- **Environment tags:** myapp:1.2.3-prod, myapp:1.2.3-staging — environment-specific config baked in (avoid if possible, prefer env vars)

```
# GitHub Actions CI/CD example
- name: Build and Push
  run: |
    docker build -t ${ env.REGISTRY }}/myapp:${ github.sha } .
    docker tag myapp:${ github.sha } myapp:latest
    docker push ${ env.REGISTRY }}/myapp:${ github.sha }
```

8.2 Container Registries

Registry	Type	Best For
Docker Hub	Public/Private cloud	Open source projects, small teams, public images
Amazon ECR	Managed, AWS-integrated	AWS deployments — ECS, EKS. IAM auth, lifecycle policies
Google GCR/Artifact Registry	Managed, GCP-integrated	GCP deployments — GKE. Binary auth, vulnerability scanning
Azure ACR	Managed, Azure-integrated	AKS deployments. Geo-replication, tasks for builds
GitHub Container Registry (GHCR)	GitHub-integrated	Open source, GitHub Actions CI/CD workflows
Harbor	Self-hosted, open source	On-premise, full control, enterprise features

9. Interview Questions

Core Concepts

Q1. What is a Docker image vs container?

→ Image: read-only layered template — app + runtime + libs. Blueprint. Container: running instance of image — isolated process with own FS, network, PID namespace. Image → Container like Class → Object

Q2. How do Docker layers work?

→ Each Dockerfile instruction creates a read-only layer. Layers are cached and shared between images saving space. Containers add thin writable layer on top. Layer cache invalidated if instruction or its inputs change

Q3. What is multi-stage build? Why use it?

→ Multiple FROM instructions in one Dockerfile. Early stages build (have compiler, Maven, etc.). Final stage copies only artifacts — no build tools in production image. Result: smaller image, smaller attack surface

Q4. What is the difference between CMD and ENTRYPOINT?

→ ENTRYPOINT: executable — not overridden by docker run args (args appended). CMD: default args to ENTRYPOINT, overridable. Best practice: ENTRYPOINT ["java"] CMD ["-jar","app.jar"]. Or ENTRYPOINT ["java","-jar","app.jar"] for fixed command

Q5. How does layer caching work? How to optimize?

→ Docker uses cache if instruction AND all previous layers unchanged. Put slow/infrequent instructions early: FROM, apt-get installs, dependency downloads. Put fast/frequent: COPY src . last. Cache busted by any file change in COPY context

Networking & Volumes

Q6. How do containers in Docker Compose communicate?

→ Compose puts all services on same bridge network by default. Services communicate by service name (DNS). app container reaches db at hostname 'db' — no IP needed. Isolated from host network

Q7. Named volume vs bind mount?

→ Named volume: Docker manages location (/var/lib/docker/volumes/). Portable, works on all OSes, best for persistent data (DB). Bind mount: maps specific host path — host-dependent, great for dev (hot reload, config)

Q8. What is --network=host?

→ Container uses host's network stack — no NAT, no port mapping. Direct access to all host ports. Slight performance benefit. Not available on Docker Desktop (Mac/Windows). Security risk — container can reach all host ports

Q9. What is the difference between EXPOSE and -p in docker run?

→ EXPOSE: documentation only — tells what port container uses internally. Does NOT publish port. -p 8080:8080 (or --publish): actually publishes host:container port mapping. Must use -p to access from outside

Security & Best Practices

Q10. Why run containers as non-root?

→ Root in container can potentially escalate to root on host via misconfigurations or vulnerabilities. Non-root user limits blast radius. Add: RUN adduser -S appuser && USER appuser. Required by many K8s pod security policies

Q11. How do you handle secrets in Docker?

→ Never in image layers (persist even if deleted). Never in plain env vars (visible in docker inspect). Options: Docker Secrets (Swarm), K8s Secrets, HashiCorp Vault, tmpfs mounts. Inject at runtime, not build time

Q12. What is distroless image?

→ Google's minimal base image — only app runtime, no shell, no package manager, no utilities. Smallest attack surface. e.g., gcr.io/distroless/java21. Can't exec into running container (security benefit but harder debugging)

Q13. How to reduce image size?

→ Use Alpine or distroless base. Multi-stage builds. Combine RUN commands (fewer layers). .dockerignore to exclude build artifacts. Remove apt cache: apt-get clean && rm -rf /var/lib/apt/lists/*

Operations

Q14. How do you debug a running container?

→ docker logs -f container (live logs). docker exec -it container bash (interactive shell). docker inspect container (full config). docker stats (CPU/mem). docker cp to extract files. If no shell (distroless): use ephemeral debug container

Q15. What is .dockerignore?

→ Like .gitignore for Docker build context. Excludes files from being sent to daemon. Exclude: target/, .git/, *.md, node_modules, local config. Reduces build context size = faster builds

Q16. What is docker system prune?

→ Removes: stopped containers, unused networks, dangling images, unused build cache. -a flag also removes all unused images (not just dangling). -v also removes unused volumes. Use to reclaim disk space

Q17. How does health check work in Docker?

→ HEALTHCHECK instruction defines command to test container health. Docker runs it periodically. Status: starting → healthy/unhealthy. Compose depends_on condition: service_healthy waits for healthy status. K8s has its own probe system (ignores Docker HEALTHCHECK)

Q18. Explain Docker's copy-on-write (CoW) mechanism?

→ All containers share image layers (read-only). When container writes to a file from image layer, that file is copied to container's writable layer (CoW). Only modified files are in writable layer. Efficient — many containers from same image share most data

Q19. What is the difference between docker stop and docker kill?

→ stop: sends SIGTERM — graceful shutdown. App can clean up, finish requests, close connections. Waits 10s (configurable), then SIGKILL. kill: sends SIGKILL immediately — no cleanup. Use stop in production, kill only if stop hangs

Q20. How do you handle container logging?

→ Docker captures stdout/stderr as logs. Log drivers: json-file (default), syslog, journald, fluentd, awslogs. In production: use fluentd or awslogs driver to ship to centralized logging (ELK, CloudWatch). Never log to files inside container — use stdout

Quick Fire

Q21. What is Docker context?

→ Files sent to Docker daemon during build. All files in directory (respecting .dockerignore). Large context = slow build. Solution: .dockerignore or build from subdirectory

Q22. What is a dangling image?

→ Image with no tag — shows as :. Created when image rebuilt with same tag. Remove with docker image prune. Also created from intermediate build stages

Q23. How do you share environment between containers in Compose?

→ Environment variables per service in compose.yml. env_file: .env for shared values. Shared volume for config files. Service can read another's env via shared file or external config service

Q24. What is BuildKit?

→ Modern Docker build engine. Parallel layer building, better caching, secrets during build (--mount=type=secret), SSH agent forwarding. Enable: DOCKER_BUILDKIT=1 or in daemon.json. Default in Docker 23+

Q25. What is docker save and docker load?

→ save: export image to tar file — docker save myapp:1.0 | gzip > myapp.tar.gz. load: import image from tar — docker load < myapp.tar.gz. Transfer images without registry — air-gapped environments